

Practice Assignment 6

CHAT APPLICATION

Continuing from the previous lab exercises, the library system now wants to integrate an additional chat channel with readers and create an environment for readers to interact with each other. Please help the library implement this requirement based on the following descriptions:

LEARNING OBJECTIVES:

- Understand and configure SignalR in ASP.NET Core
- Build a realtime chat app (broadcast → group → private)
- Save message history to SQL Server database
- Separate processing layers (Hub – Service – Model – Repository)

Exercise 1: Create a basic real-time chat application (Broadcast)

Requirements:

- Create an **ASP.NET Core Web API** project.
- Integrate a **SignalR Hub** (ChatHub) with a method SendMessage(string user, string message).
- Create a **Razor View** or **MVC View** to connect to the Hub for sending and receiving messages in real-time.
- Messages sent from one client should appear on all connected clients.

Objective: Understand the **Hub–Client communication** and **broadcast messaging** with SignalR.

Exercise 2: Add group chat functionality

Requirements:

- Create an API endpoint allowing clients to **join or leave a room** (JoinRoom, LeaveRoom).
- Hub should have a method SendGroupMessage(string roomName, string user, string message).
- Razor View should display a list of rooms and allow users to send messages within each room.

Objective: Practice **SignalR Groups** and integrating API with UI for room management.

Exercise 3: Add private chat functionality

Requirements:

- Create an API to manage a **list of online users** (username + connectionId).
- Hub should support SendPrivateMessage(string receiverConnectionId, string user, string message).
- Razor View or a separate view should allow selecting a recipient for private messages.

Objective: Implement **private messaging** via SignalR, combined with **API + View**.

Exercise 4: Store chat history in SQL Server (EF Core)

Requirements:

- Design a sample database:

Example table Messages

Field	Type	Description
Id	INT PK	Primary key
SenderId	INT FK	Sender
ReceiverId	INT FK NULL	Receiver, NULL if broadcast/group
RoomName	NVARCHAR(50) NULL	Room name if group

MessageBody	NVARCHAR(MAX)	Message content
MessageType	TINYINT	0=broadcast,1=group,2=private
CreatedAt	DATETIME	Timestamp

- Use **EF Core** to store messages sent via the Hub.
- Create an API to **fetch chat history** and display it in the view when the page loads.

Objective: Learn how to combine **SignalR + EF Core + SQL Server + API**.

Exercise 5: Advanced – Complete the application

Requirements:

- Add **authentication** for users (Identity or token) to connect to the Hub.
- Display **chat history by room or user** in the view.
- Show **notifications when users join/leave rooms** in real-time.
- Optimize architecture: separate **Service** and **Repository layers** for API and Hub.
- Log messages to a **file or HistoryLog table** in the database.

Objective: Apply **best practices** in building a real-time chat application on **API + View**.

Advantages of this approach:

- Students practice both **API development** (endpoints, CRUD, authentication) and **SignalR real-time features**.
- Views can be **integrated within the same project** or built **separately** to call the API + Hub.
- Prepares students for real-world scenarios: Web API + SPA or MVC front-end.

Some suggestions for completing the assignment

(Important note: this is for reference only. To build a complete chat application, you need to apply your own programming thinking and rely on the knowledge you have learned.)

1. For group chat, there is no need to store chat groups Id or members in the group into the database; a chat group is created when users connect to the hub and will be deleted after leaving the hub.
2. Package need:
 - a. dotnet add package Microsoft.AspNetCore.SignalR
 - b. dotnet add package Microsoft.EntityFrameworkCore.SqlServer
 - c. dotnet add package Microsoft.EntityFrameworkCore.Tools
 - d. dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
3. Create chatHub

```
using Microsoft.AspNetCore.SignalR;

public class ChatHub : Hub
{
    private readonly ChatDbContext _context;
    public ChatHub(ChatDbContext context)
    {
        _context = context;
    }

    // Broadcast
    public async Task SendMessage(string user, string message)
    {
        await Clients.All.SendAsync("ReceiveMessage", user, message);

        var sender = _context.Users.FirstOrDefault(u => u.Username == user);
        _context.Messages.Add(new Message { SenderId = sender?.Id ?? 0, MessageBody = message, MessageType = 0 });
        await _context.SaveChangesAsync();
    }

    // Group chat
    public async Task JoinRoom(string roomName)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, roomName);
    }

    public async Task LeaveRoom(string roomName)
    {
        await Groups.RemoveFromGroupAsync(Context.ConnectionId, roomName);
    }

    public async Task SendGroupMessage(string roomName, string user, string message)
    {
        await Clients.Group(roomName).SendAsync("ReceiveGroupMessage", user, message, roomName);

        var sender = _context.Users.FirstOrDefault(u => u.Username == user);
        _context.Messages.Add(new Message { SenderId = sender?.Id ?? 0, MessageBody = message, RoomName = roomName, MessageType = 1 });
        await _context.SaveChangesAsync();
    }
}
```

```
// Private chat
public async Task SendPrivateMessage(string receiverConnectionId, string user, string message)
{
    await Clients.Client(receiverConnectionId).SendAsync("ReceivePrivateMessage", user, message);

    var sender = _context.Users.FirstOrDefault(u => u.Username == user);
    _context.Messages.Add(new Message { SenderId = sender?.Id ?? 0, MessageBody = message, MessageType = 2 });
    await _context.SaveChangesAsync();
}
```

4. Program.cs

```
builder.Services.AddSignalR();
```

```
app.MapHub<ChatHub>("/chathub");
```

5. Razor View

Views/Chat/Index.cshtml:

```
@{
    ViewData["Title"] = "RealTime Chat";
}

<h2>Broadcast Chat</h2>
<input id="userInput" placeholder="Your name" />
<input id="messageInput" placeholder="Message" />
<button id="sendBtn">Send</button>

<div id="messages"></div>

<script src="https://cdnjs.cloudflare.com/ajax/libs/microsoft-signalr/8.0.0/signalr.min.js"></script>
<script>
    const connection = new signalR.HubConnectionBuilder()
        .withUrl("/chathub")
        .build();

    connection.on("ReceiveMessage", (user, message) => {
        const msg = `${user}: ${message}`;
        document.getElementById("messages").innerHTML += msg + "<br>";
    });

    connection.start().catch(err => console.error(err));

    document.getElementById("sendBtn").addEventListener("click", () => {
        const user = document.getElementById("userInput").value;
        const message = document.getElementById("messageInput").value;
        connection.invoke("SendMessage", user, message);
    });
</script>
```

Similarly, create views for both group chat and private chat

6. API endpoints (Optional)

```
[ApiController]
[Route("api/[controller]")]
public class ChatController : ControllerBase
{
    private readonly ChatDbContext _context;
    public ChatController(ChatDbContext context) { _context = context; }

    [HttpGet("history")]
    public async Task<IActionResult> GetHistory() => Ok(await _context.Messages.ToListAsync());
}
```

Used to load message history on view.

7. Authentication (JWT Example)

```

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new Microsoft.IdentityModel.Tokens.TokenValidationParameters
    {
        ValidateIssuer = false,
        ValidateAudience = false,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("YourSecretKey12345"))
    };
    options.Events = new JwtBearerEvents
    {
        OnMessageReceived = context =>
        {
            var accessToken = context.Request.Query["access_token"];
            var path = context.HttpContext.Request.Path;
            if (!string.IsNullOrEmpty(accessToken) && path.StartsWithSegments("/chathub"))
            {
                context.Token = accessToken;
            }
            return Task.CompletedTask;
        }
    };
});

builder.Services.AddAuthorization();
app.UseAuthentication();
app.UseAuthorization();

```

Use JWT or Identity. When client connects to hub, send token:

```

.withUrl("/chathub", { accessTokenFactory: () => token })

```

Hub has [Authorize] on class or method.